



UNIVERSIDAD DE BURGOS
ESCUELA POLITÉCNICA SUPERIOR
Grado en Ingeniería Informática



TFG del Grado en Ingeniería
Informática

Detección del parásito Varroa
Destructor mediante redes
neuronales



Presentado por Manuel Meléndez Ortega
en Universidad de Burgos — 10 de junio
de 2026

Tutor: Carlos Cambra Baseca



UNIVERSIDAD DE BURGOS
ESCUELA POLITÉCNICA SUPERIOR
Grado en Ingeniería Informática



D. Carlos Cambra Baseca, profesor del departamento de Digitalización, área de Ciencia de la Computación e Inteligencia Artificial.

Expone:

Que el alumno D. Manuel Meléndez Ortega, con DNI 26279201E, ha realizado el Trabajo Final de Grado en Ingeniería Informática titulado Detección del parásito Varroa Destructor mediante redes neuronales.

Y que dicho trabajo ha sido realizado por el alumno bajo la dirección del que suscribe, en virtud de lo cual se autoriza su presentación y defensa.

En Burgos, 10 de junio de 2026

Vº. Bº. del Tutor:

D. Carlos Cambra Baseca

Resumen

La apicultura actual se enfrenta a una grave amenaza debido al ácaro *Varroa destructor*, un parásito que debilita las colonias y puede provocar su colapso si no se detecta a tiempo. Tradicionalmente, la monitorización de esta plaga se realiza mediante la inspección manual de bandejas sanitarias, un proceso laborioso, poco escalable y susceptible al error humano.

Este Trabajo de Fin de Grado propone el desarrollo de un sistema automático basado en visión por computador y modelos de aprendizaje profundo para la detección y conteo de este ácaro en imágenes fotográficas. La solución desarrollada se integra en una aplicación web intuitiva que permite a los apicultores subir imágenes de sus colmenas y obtener una estimación visual y numérica de la infestación. El objetivo de esta herramienta es modernizar el control sanitario apícola, reduciendo el tiempo de inspección manual y facilitando la toma de decisiones para proteger la viabilidad y sostenibilidad de las explotaciones.

Descriptores

Inteligencia artificial, *Varroa destructor*, visión por computador, aprendizaje profundo, apicultura, automatización agrícola.

Abstract

Modern beekeeping faces a severe threat from the *Varroa destructor* mite, a parasite that weakens colonies and can cause their collapse if not detected in time. Traditionally, monitoring this pest involves the manual inspection of sticky boards, a labor-intensive, unscalable, and error-prone process.

This Final Degree Project proposes the development of an automated system based on computer vision and deep learning models for the detection and counting of this mite in photographic images. The developed solution is integrated into an intuitive web application that allows beekeepers to upload images of their hives and obtain a visual and numerical estimate of the infestation. The objective of this tool is to modernize beekeeping health control, reducing manual inspection time and facilitating decision-making to protect the viability and sustainability of apiaries.

Keywords

Artificial intelligence, *Varroa destructor*, computer vision, deep learning, beekeeping, agricultural automation.

Índice general

Índice general	iii
Índice de figuras	v
Índice de tablas	vi
1. Introducción	1
2. Objetivos del proyecto	3
2.1. Objetivos marcados por los requisitos del software	3
2.2. Objetivos de carácter técnico	4
3. Conceptos teóricos	5
3.1. Redes neuronales	5
3.2. Redes neuronales convolucionales (CNN)	16
3.3. Detectores basados en R-CNN	24
3.4. Métricas de evaluación	31
4. Técnicas y herramientas	35
4.1. Entorno de desarrollo y tecnologías	35
4.2. Arquitectura del modelo	36
4.3. Preparación del dataset	37
4.4. Proceso de entrenamiento y optimización	39
4.5. Despliegue e interfaz de usuario	41
5. Aspectos relevantes del desarrollo del proyecto	43
5.1. Metodología y ciclo de vida	43
5.2. Arquitectura del sistema y desarrollo	45

5.3. Retos técnicos y limitaciones	45
5.4. Evaluación y resultados del modelo	47
6. Trabajos relacionados	49
7. Conclusiones y líneas de trabajo futuras	51
7.1. Conclusiones	51
7.2. Posibilidades de mejora y líneas futuras	52
Bibliografía	53

Índice de figuras

3.1. Ejemplo de una red neuronal simple.	7
3.2. Función de activación sigmoidal.	9
3.3. Función de activación Tanh.	9
3.4. Función de activación ReLU.	10
3.5. Función de pérdida Smooth L1.	13
3.6. Algoritmo de optimización SGD sin y con momentum.	15
3.7. Ejemplo del flujo de trabajo de una CNN para detectar el número 2.	18
3.8. Ejemplo del proceso de convolución.	19
3.9. Visualización de la convolución en una imagen RGB.	20
3.10. Ejemplo de uso del padding.	21
3.11. Uso de Max Pooling y Average Pooling.	22
3.12. Ejemplo simple de flatten.	23
3.13. Ejemplo de funcionamiento de una R-CNN.	25
3.14. Flujo de funcionamiento de un Fast R-CNN.	27
3.15. Simplificación de un Faster R-CNN.	28
3.16. Ejemplo de un RPN seccionando una imagen.	29
3.17. Ejemplo de uso de NMS.	30
4.1. Comparativa de una CNN con ResNet.	36
4.2. Vista simplificada del funcionamiento de FPN.	37
4.3. Ejemplo de resultado del uso de la biblioteca albumentation. . .	38
4.4. Comparativa entrenamiento normal y transfer learning.	39
4.5. Ejemplo de uso de StepLR.	40
5.1. Flujo de trabajo de CRISP-DM.	44
5.2. Imagen del dataset mostrando la falta de resolución.	46

Índice de tablas

3.1. Comparativa y evolución de las arquitecturas de detección de objetos.	31
4.1. Distribución final del conjunto de datos tras la aumentación (partiendo de 429 imágenes originales y 1020 generadas sintéticamente).	38

1. Introducción

En los últimos años, el sector agrario ha comenzado a incorporar de forma progresiva herramientas procedentes del ámbito de la informática con el fin de optimizar procesos, reducir costes y mejorar la sostenibilidad de las explotaciones. Estas aplicaciones permiten automatizar tareas repetitivas, tratar grandes volúmenes de datos y apoyar la toma de decisiones en entornos donde las variables biológicas y ambientales dificultan el uso de métodos deterministas propios de la industria. A diferencia del sector manufacturero, la agricultura presenta una elevada variabilidad y condiciones difíciles de controlar, lo que ha impulsado el uso de técnicas más flexibles como las que proporciona la Inteligencia Artificial.

Dentro de este campo, la visión por computador se ha consolidado como una de las tecnologías más prometedoras para el análisis automático de imágenes agrícolas, permitiendo identificar plagas, estimar producciones o vigilar el estado de los cultivos sin intervención humana directa. Los sistemas que emplean aprendizaje profundo [13] destacan por su capacidad para extraer características relevantes a partir de datos visuales y adaptarse a escenarios complejos donde los métodos tradicionales resultan insuficientes.

En el ámbito de la apicultura, una de las amenazas más significativas para la supervivencia de las colmenas es el ácaro *Varroa Destructor* [25]. Este parásito afecta tanto a larvas como a abejas adultas, debilitando progresivamente a la colonia y pudiendo llegar a provocar su colapso [16] si no se detecta a tiempo. Tradicionalmente, el control de la varroa se realiza mediante inspección manual de bandejas sanitarias situadas en la base de las colmenas [4]. Sin embargo, este método resulta laborioso, poco escalable y susceptible al error humano, especialmente cuando se manejan grandes volúmenes de colmenas o imágenes.

El presente proyecto propone el desarrollo de un sistema automático capaz de detectar ejemplares de *Varroa Destructor* en imágenes de estas bandejas sanitarias mediante técnicas de visión artificial [1]. Para ello, se emplea un modelo de aprendizaje profundo especializado en la detección de objetos, junto con un proceso de entrenamiento adaptado al conjunto de datos disponible. El sistema se complementa con una aplicación web que permite al usuario cargar imágenes y obtener una estimación visual y numérica del número de varroas detectadas, facilitando su uso en contextos reales.

A largo plazo, este tipo de herramientas puede contribuir a mejorar la gestión de las colmenas, reducir el tiempo dedicado a la inspección manual y aumentar la precisión en la monitorización sanitaria de la colmena.

2. Objetivos del proyecto

En este capítulo se definen los objetivos que guían el desarrollo del presente Trabajo de Fin de Grado. Con el fin de clarificar el alcance del sistema que se quiere construir, los objetivos se han dividido en dos grandes bloques: por un lado, los objetivos marcados por los requisitos del software, que describen lo que el sistema debe ser capaz de ofrecer al usuario; y por otro, los objetivos de carácter técnico, que recogen las capacidades internas necesarias para que la solución funcione de manera fiable y eficiente.

2.1. Objetivos marcados por los requisitos del software

- **Desarrollar una página web que permita a cualquier usuario cargar imágenes y probar el sistema de detección.**

El sistema debe incluir una interfaz web sencilla desde la cual el usuario pueda seleccionar una o varias imágenes de bandejas blancas procedentes de colmenas y enviarlas al servidor para su procesamiento. Esta página web actuará como punto de acceso al modelo de detección, evitando que el usuario tenga que interactuar directamente con código o herramientas de línea de comandos.

- **Crear un modelo capaz de identificar correctamente ejemplares de *Varroa Destructor* en imágenes tomadas sobre un fondo blanco.**

El software debe ser capaz de analizar las imágenes recibidas y localizar las varroas presentes sobre el fondo blanco de la bandeja sanitaria [4]. El objetivo es que el sistema detecte la mayor parte de los parási-

tos visibles, diferenciándolos de otros elementos como restos de cera, suciedad u otros insectos que puedan aparecer en la imagen.

- **Proporcionar estadísticas basadas en los resultados de detección que faciliten la interpretación del nivel de infestación.**

Además de mostrar el resultado individual de cada imagen, el sistema debe ser capaz de calcular información agregada a partir de las detecciones realizadas, como el número total de varroas detectadas por imagen o por lote de imágenes. Estas estadísticas tienen como finalidad ayudar al usuario a interpretar el estado de las colmenas y a tomar decisiones informadas sobre posibles tratamientos o medidas de control.

2.2. Objetivos de carácter técnico

- **Alcanzar un rendimiento mínimo aceptable en métricas estándar de detección de objetos.**

El modelo de detección debe obtener unos resultados razonables en métricas ampliamente utilizadas en visión por computador, como la precisión (*precision*), el *recall*, la intersección sobre unión (IoU) [23] y la media de la precisión promedio (mAP@0.5) [5]. Estos indicadores permitirán evaluar la calidad del sistema y comprobar si su rendimiento es suficiente para apoyar la toma de decisiones.

- **Asegurar que el modelo sea capaz de detectar varroas incluso en condiciones difíciles.**

El sistema debe ser capaz de identificar varroas cuando su tamaño en la imagen es reducido, cuando el contraste con el fondo no es óptimo o cuando aparecen parcialmente ocultas entre otros residuos de la bandeja. Este objetivo técnico está orientado a mejorar la robustez del modelo frente a situaciones reales, donde las imágenes no siempre presentan condiciones ideales.

- **Garantizar que el tiempo de inferencia por imagen sea suficientemente reducido para permitir un uso fluido de la aplicación web.**

El tiempo que transcurre desde que el usuario envía una imagen hasta que recibe el resultado debe ser razonable para que la herramienta resulte práctica en un entorno real. Este objetivo implica ajustar el modelo y la forma en que se ejecuta para que la detección pueda realizarse en un plazo de tiempo aceptable en hardware convencional.

3. Conceptos teóricos

En este capítulo se presentan los conceptos teóricos necesarios para comprender el sistema de detección de varroa desarrollado en este proyecto. Dado que la solución se basa en técnicas de visión por computador y aprendizaje profundo [13, 7], se introducen los elementos esenciales de las redes neuronales, las redes neuronales convolucionales (CNN) y los modelos de detección de objetos utilizados actualmente.

3.1. Redes neuronales

Las redes neuronales artificiales son modelos matemáticos basados en el funcionamiento del cerebro humano, formados por unidades llamadas neuronas, organizadas en capas. Cada neurona recibe valores de entrada, los combina mediante parámetros ajustables y produce una salida que se transmite a la siguiente capa.

3.1.1 Funcionamiento interno de una red neuronal artificial

La finalidad de una red neuronal es aprender patrones presentes en los datos mediante el ajuste progresivo de los parámetros internos que controlan el flujo de información. Estos parámetros, conocidos como *pesos* y *sesgos*, determinan cómo responde cada neurona ante los estímulos que recibe.

Las redes neuronales se estructuran en **capas**, formadas por un conjunto de **neuronas**. Cada neurona realiza una operación sencilla pero fundamental: calcula una combinación ponderada de sus entradas y aplica a dicho resultado

una función de activación. Formalmente, para una neurona j , la operación puede expresarse como:

$$z_j = \sum_i w_{ij} x_i + b_j$$

donde x_i son las entradas provenientes de la capa anterior, w_{ij} es el peso que conecta la entrada i con la neurona j , y b_j es el sesgo asociado.

El valor resultante z_j se transforma mediante una función de activación f , produciendo la salida de la neurona:

$$a_j = f(z_j)$$

Este mecanismo —suma ponderada seguida de activación— se repite de capa en capa, permitiendo que la información fluya desde los datos de entrada hasta la predicción final realizada por la red.

Propagación hacia adelante (forward pass)

El proceso mediante el cual una red neuronal transforma una entrada en una salida se conoce como *propagación hacia adelante*. Cada capa recibe la representación generada por la anterior y la transforma aplicando sus propios pesos y funciones de activación. De forma conceptual, el proceso puede dividirse en tres etapas:

- **Capa de entrada:** recibe los datos originales, por ejemplo, una imagen.
- **Capas ocultas:** generan representaciones internas cada vez más complejas combinando patrones aprendidos en capas previas.
- **Capa de salida:** devuelve la predicción final, que puede ser una clasificación o, en modelos de detección, coordenadas y probabilidades asociadas a los objetos encontrados.

Estas representaciones no se diseñan explícitamente, sino que se aprenden durante el entrenamiento.

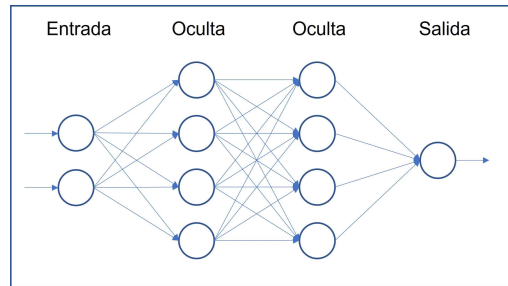


Figura 3.1: Ejemplo de una red neuronal simple.

Proceso de aprendizaje

Para que la red neuronal mejore con el tiempo, los pesos deben ajustarse en función de los errores cometidos. Este ajuste se realiza mediante un proceso denominado *retropropagación del error* (*backpropagation*) [26]. De forma simplificada, el mecanismo consiste en:

1. Generar una predicción mediante la propagación hacia adelante.
2. Comparar la predicción con la etiqueta real mediante una función de pérdida.
3. Calcular cómo afecta cada peso al error total.
4. Ajustar los pesos en la dirección que reduce ese error.

Este proceso permite que la red aprenda progresivamente patrones presentes en los datos. Una explicación más detallada se desarrolla más adelante en la Sección 3.1.5.

Conectividad entre neuronas

En las redes neuronales clásicas de tipo *feedforward*, cada neurona de una capa se conecta con todas las neuronas de la capa siguiente. Este tipo de arquitectura, denominada *capa densa* o *fully connected*, sirve como base conceptual para comprender arquitecturas más complejas, como las redes convolucionales utilizadas en este proyecto.

En una capa densa:

- Cada neurona recibe todas las salidas de la capa anterior.

- Los pesos determinan la importancia de cada conexión.
- El entrenamiento refuerza las conexiones útiles y debilita las irrelevantes.

Gracias a este mecanismo de comunicación, las redes neuronales son capaces de modelar relaciones complejas entre las variables de entrada y las salidas deseadas.

3.1.2 Funciones de activación

Las funciones de activación determinan cómo transforma su salida cada neurona. Permiten que la red neuronal aprenda relaciones que no son puramente lineales, algo necesario para representar patrones complejos presentes en las imágenes. A continuación se describen las funciones más utilizadas y la empleada en este proyecto.

Sigmoid

La función Sigmoid transforma cualquier valor real en un número entre 0 y 1:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Ventajas:

- La salida está acotada entre 0 y 1.

Desventajas:

- Produce gradientes muy pequeños en valores extremos, lo que dificulta el aprendizaje en redes profundas.

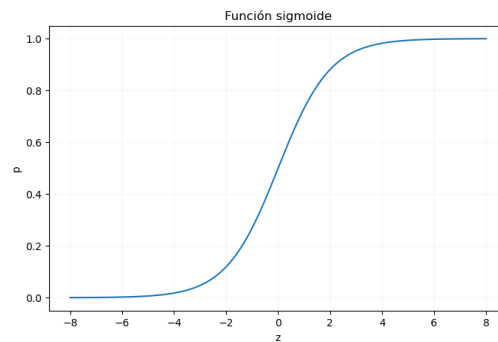


Figura 3.2: Función de activación sigmoideal.

Tanh

La función Tanh devuelve valores entre -1 y 1:

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

Ventajas:

- Sus valores están centrados alrededor de cero, lo que suele favorecer el aprendizaje.

Desventajas:

- Al igual que Sigmoid, presenta gradientes pequeños lejos del origen.

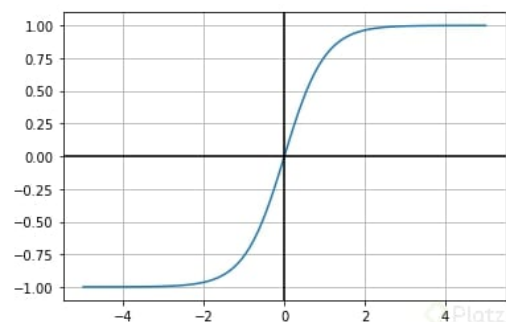


Figura 3.3: Función de activación Tanh.

ReLU

La unidad lineal rectificadora [17] se define como:

$$\text{ReLU}(x) = \max(0, x)$$

Ventajas:

- Es simple de calcular.
- No sufre el problema de gradientes pequeños en valores positivos.
- Funciona bien en modelos de visión por computador.

Desventajas:

- Puede dejar neuronas con salida cero de manera permanente si reciben valores negativos de forma continua.

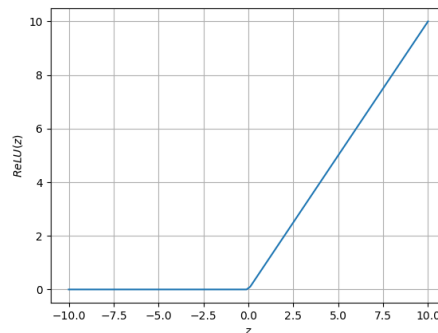


Figura 3.4: Función de activación ReLU.

Variantes de ReLU

Existen versiones modificadas, como Leaky ReLU o ELU, que introducen pequeñas modificaciones en la parte negativa para evitar que las neuronas queden inactivas. No se utilizan en este proyecto.

Función utilizada en este proyecto

El modelo empleado utiliza ReLU como función de activación en sus capas internas. Esta elección forma parte de la implementación estándar de PyTorch [20] y ofrece un equilibrio adecuado entre simplicidad y rendimiento en tareas de detección de objetos.

3.1.3 Épocas, tamaño de *batch* y problemas de ajuste

Durante el entrenamiento de una red neuronal, el conjunto de datos no se procesa en una única pasada, sino a lo largo de varias repeticiones llamadas *épocas*. En cada época, el modelo observa todas las muestras del conjunto de entrenamiento una vez, ajustando progresivamente sus pesos.

Las muestras no se procesan individualmente, sino en grupos denominados *batches*. El uso de lotes permite aprovechar las operaciones vectorizadas, acelerando los cálculos y proporcionando gradientes más estables que los que se obtendrían procesando una sola muestra cada vez.

El número de épocas tiene un impacto directo en la calidad del aprendizaje:

- **Pocas épocas** pueden producir *falta de ajuste* (*underfitting*), situación en la que el modelo no ha aprendido suficientemente los patrones del conjunto de entrenamiento.
- **Demasiadas épocas** pueden generar *sobreajuste* (*overfitting*), en cuyo caso el modelo aprende detalles excesivamente específicos del conjunto de entrenamiento y pierde capacidad de generalización.

Para identificar estos problemas se utiliza un *conjunto de validación* separado del entrenamiento. Durante el proceso, se monitoriza la evolución de la pérdida en ambas particiones:

- Si la pérdida de entrenamiento disminuye mientras que la de validación deja de mejorar o aumenta, es un indicio claro de sobreajuste;
- Si ambas pérdidas permanecen altas, puede existir una falta de ajuste.

El entrenamiento del modelo utilizado en este proyecto sigue este enfoque, evaluando periódicamente la pérdida de validación para determinar si el aprendizaje progresa adecuadamente y si es necesario ajustar los hiperparámetros.

3.1.4 Función de pérdida

La función de pérdida (o *loss*) es el mecanismo que permite medir cuánto se están equivocando las predicciones de la red neuronal respecto a los valores esperados. A cada ejemplo del conjunto de entrenamiento le corresponden unas etiquetas reales (clase correcta, posición de la caja delimitadora, ...). El modelo genera unas predicciones y la función de pérdida convierte la diferencia entre ambas en un único número real.

Durante el entrenamiento, el objetivo es minimizar ese número: si la pérdida es alta, el modelo está cometiendo grandes errores; si la pérdida se acerca a cero, las predicciones se parecen mucho a las etiquetas.

En la detección de objetos, como la que se realiza en este proyecto, suele descomponerse la pérdida en dos términos principales:

$$L = L_{\text{cls}} + L_{\text{bbox}}$$

donde:

- L_{cls} mide el error al asignar la clase correcta (en este caso, “varroa” o fondo).
- L_{bbox} mide el error en la posición y tamaño de las cajas delimitadoras predichas.

Pérdida de clasificación Para la parte de clasificación se utiliza habitualmente la entropía cruzada (*cross-entropy*). Sea C el número de clases (en nuestro caso $C = 2$ contando el fondo), y_c la etiqueta real en formato *one-hot* (vale 1 para la clase correcta y 0 para el resto) y $\hat{y}_c$ la probabilidad predicha por la red para la clase c . La pérdida de entropía cruzada se escribe como:

$$L_{\text{cls}} = - \sum_{c=1}^C y_c \log \hat{y}_c$$

Si el modelo asigna una probabilidad alta a la clase correcta ($\hat{y}_c$ cercano a 1), el término $-\log \hat{y}_c$ es pequeño y la pérdida disminuye. Por el contrario, si la probabilidad de la clase correcta es baja, el valor de $-\log \hat{y}_c$ crece y la pérdida penaliza más ese error.

Pérdida de regresión para las cajas delimitadoras Además de decidir si en una región hay o no una varroa, el modelo debe aprender a ajustar la caja delimitadora que la encuadra. Para ello se plantea un problema de regresión sobre las coordenadas de la caja.

En Faster R-CNN [22] se utiliza la llamada función *Smooth L1* para esta tarea, porque combina el buen comportamiento de una pérdida cuadrática (cuando el error es pequeño) con la robustez de una pérdida lineal (cuando el error es grande). Si denotamos por x la diferencia entre la coordenada real y la predicha, la función Smooth L1 se define como:

$$\text{SmoothL1}(x) = \begin{cases} 0,5 x^2 & \text{si } |x| < 1 \\ |x| - 0,5 & \text{en otro caso} \end{cases}$$

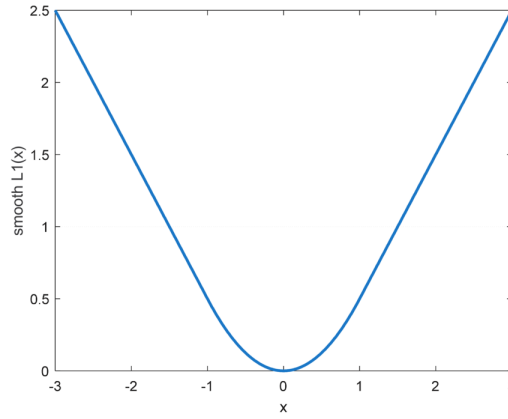


Figura 3.5: Función de pérdida Smooth L1.

De forma simplificada, la pérdida de regresión sobre una caja se puede escribir como:

$$L_{\text{bbox}} = \frac{1}{N_{\text{pos}}} \sum_{i=1}^{N_{\text{pos}}} \sum_{k \in \{x, y, w, h\}} \text{SmoothL1}(t_{i,k} - \hat{t}_{i,k})$$

donde N_{pos} es el número de propuestas positivas (aquellas que realmente contienen una varroa), $t_{i,k}$ son los parámetros reales de la caja (posición y tamaño) y $\hat{t}_{i,k}$ los valores predichos por el modelo para cada componente k .

En resumen, la pérdida total L combina el error de clasificación y el de localización. Durante el entrenamiento, PyTorch [20] calcula internamente

estos términos y ajusta los pesos de la red para reducirlos, de manera que el modelo vaya mejorando tanto en decidir dónde hay varroa como en encuadrarla correctamente.

3.1.5 Aprendizaje del modelo: retropropagación, tasa de aprendizaje y optimización

El proceso de aprendizaje de una red neuronal se basa en tres elementos fundamentales: la retropropagación, la tasa de aprendizaje y el algoritmo de optimización. Estos componentes actúan conjuntamente para permitir que el modelo mejore su desempeño a lo largo del entrenamiento.

Retropropagación. Tras la propagación hacia adelante, el modelo genera una predicción que se compara con la etiqueta real mediante la función de pérdida. La diferencia obtenida permite cuantificar el error, pero todavía no indica cómo corregir los pesos. Para ello se emplea la **retropropagación del error** (*backpropagation*), que aplica la regla de la cadena para calcular el gradiente de la pérdida respecto a cada parámetro de la red.

Dado un peso w , la retropropagación calcula:

$$\frac{\partial L}{\partial w}$$

que representa cómo varía la pérdida si ese peso se modifica ligeramente. Si el valor es positivo, significa que el peso contribuye a aumentar la pérdida; si es negativo, contribuye a disminuirla. Este análisis se realiza para todos los pesos y sesgos del modelo.

Tasa de aprendizaje. El gradiente indica la dirección en la que deben ajustarse los pesos, pero no determina la magnitud del cambio. Para controlar el tamaño de cada actualización se introduce la **tasa de aprendizaje** (η). Una versión simple de la regla de actualización es:

$$w := w - \eta \frac{\partial L}{\partial w}$$

La elección de η resulta determinante:

- Si η es demasiado grande, los pesos pueden modificarse de forma excesiva, haciendo que la pérdida oscile o diverja.

- Si η es demasiado pequeña, el aprendizaje se vuelve lento y el modelo puede quedar atrapado en soluciones poco adecuadas.

En modelos preentrenados como Faster R-CNN suele emplearse una tasa de aprendizaje baja, ya que solo se ajustan parcialmente los pesos para adaptarlos a una nueva tarea (detección de varroa en este caso).

Algoritmos de optimización. Los optimizadores son los encargados de aplicar de manera eficaz las actualizaciones de los pesos a partir de los gradientes calculados. El método utilizado en este proyecto es **SGD con momento** [24, 21], ampliamente empleado en visión por computador por su estabilidad.

El concepto de **momento** introduce una “memoria” del gradiente de iteraciones anteriores. En lugar de actualizar el peso únicamente en función del gradiente actual, se acumula una parte del gradiente previo para suavizar la trayectoria del aprendizaje. La actualización se formula como:

$$v := \mu v + \eta \frac{\partial L}{\partial w} \quad w := w - v$$

donde v es la velocidad acumulada y μ es el coeficiente de momento. Esta técnica ayuda a evitar oscilaciones y acelera la convergencia en direcciones consistentes.

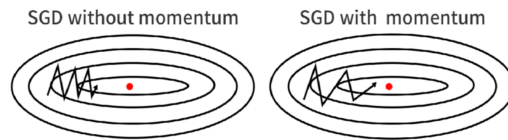


Figura 3.6: Algoritmo de optimización SGD sin y con momentum.

Nota: El algoritmo SGD estándar (sin momento) actualiza los pesos basándose únicamente en el gradiente actual y la tasa de aprendizaje. Esto provoca que, si el modelo alcanza un valle poco profundo (mínimo local) donde el gradiente se vuelve cero, las actualizaciones se detengan prematuramente, quedando el aprendizaje atascado.

Relación entre los tres componentes. Los conceptos anteriores no actúan de forma independiente: forman un ciclo continuo de mejora. La retropropagación calcula los gradientes, la tasa de aprendizaje controla el

tamaño de los cambios y el optimizador aplica la actualización. Este ciclo se repite durante cada lote de entrenamiento y en todas las épocas, permitiendo que el modelo refine progresivamente su capacidad de predicción.

En conjunto, retropropagación, tasa de aprendizaje y optimización constituyen el núcleo del proceso de aprendizaje de cualquier red neuronal, y son responsables de que el modelo pueda adaptarse a la tarea específica de detección de varroa a partir de un conjunto limitado de imágenes.

3.2. Redes neuronales convolucionales (CNN)

Las redes neuronales convolucionales (*Convolutional Neural Networks*, CNN) [14, 12] son un tipo de arquitectura diseñada específicamente para trabajar con imágenes. A diferencia de las redes densas tradicionales, las CNN emplean operaciones de convolución que permiten analizar regiones locales mediante filtros compartidos. Con esta estrategia, la red puede detectar patrones como bordes o texturas y combinarlos capa a capa para generar representaciones cada vez más complejas.

Estas propiedades han convertido a las CNN en la base de la mayoría de modelos modernos de visión por computador, incluidos los detectores de objetos utilizados en este proyecto.

Motivación para el uso de CNN en visión por computador

Las redes neuronales densas, descritas en el apartado anterior, presentan varias limitaciones cuando se aplican directamente a imágenes. En primer lugar, una imagen contiene un número muy elevado de píxeles. Por ejemplo, una imagen de 640×480 píxeles tiene 307 200 valores de entrada. Conectarla a una capa densa de únicamente 1 000 neuronas requeriría más de 300 millones de pesos. Este número de parámetros resulta difícil de almacenar, muy costoso de entrenar y altamente propenso al sobreajuste.

Además del problema de tamaño, las capas densas no aprovechan la estructura espacial de las imágenes. En una imagen, los píxeles cercanos suelen formar patrones visuales coherentes: bordes, texturas o regiones homogéneas. Sin embargo, una red densa trata cada píxel como una entrada independiente, lo que hace que ignore información esencial sobre la relación entre ellos.

Las *redes neuronales convolucionales* (CNN) se proponen como solución a estas limitaciones. En lugar de aprender pesos independientes para cada píxel, las CNN utilizan *filtros* pequeños que se desplazan por la imagen para detectar patrones locales. Este mecanismo ofrece varias ventajas:

- **Reducción drástica del número de parámetros:** un mismo filtro se utiliza en todas las posiciones de la imagen, lo que permite compartir pesos.
- **Capacidad para captar relaciones espaciales:** cada filtro analiza regiones locales, detectando bordes, esquinas, texturas u otros elementos característicos.
- **Aprendizaje jerárquico:** las primeras capas identifican patrones simples, mientras que las capas profundas combinan estos patrones para reconocer estructuras más complejas.
- **Mayor robustez frente a desplazamientos:** gracias al uso de filtros compartidos y operaciones como el *pooling*, las CNN mantienen su rendimiento incluso si el objeto aparece en distintas posiciones dentro de la imagen.

Flujo general de una CNN clásica

Aunque existen múltiples variantes, la mayoría de redes neuronales convolucionales comparten un mismo flujo de procesamiento. Este esquema resume cómo una imagen se transforma progresivamente en una representación útil para tareas como la clasificación:

1. **Entrada de la imagen.** La imagen, normalmente en formato RGB, se interpreta como un tensor tridimensional.
2. **Extracción de características mediante convoluciones.** Varias capas convolucionales aplican filtros que detectan patrones locales como bordes o texturas.
3. **Reducción espacial mediante pooling.** Algunas capas incorporan operaciones de pooling que reducen la resolución de los mapas de características y aportan invarianza a pequeñas variaciones espaciales.
4. **Aplanado (*flatten*).** Las características finales se convierten en un vector unidimensional para poder conectarlas con capas densas.

5. **Capas densas y salida.** Una o varias capas completamente conectadas combinan la información obtenida y generan la predicción final.

De forma simplificada, el flujo general puede expresarse como:

Imagen $\rightarrow$ Convolución $\rightarrow$ Pooling $\rightarrow$ Flatten $\rightarrow$ Capas densas

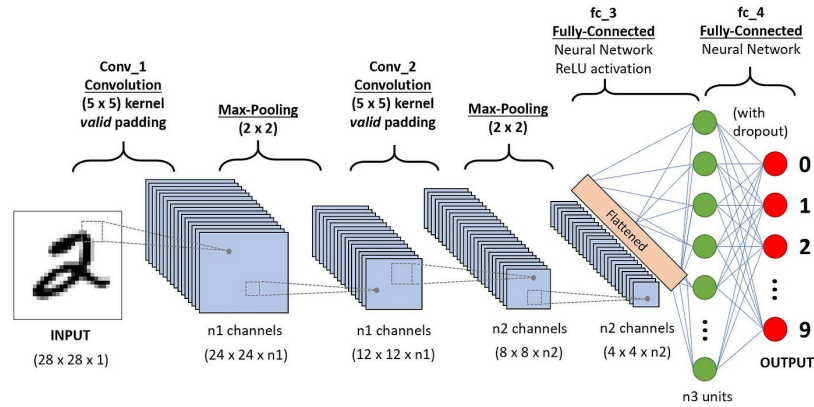


Figura 3.7: Ejemplo del flujo de trabajo de una CNN para detectar el número 2.

Este esquema servirá como base para entender posteriormente por qué las CNN clásicas no son suficientes para la detección de objetos y cómo arquitecturas modernas como Faster R-CNN superan estas limitaciones.

Operación de convolución

Dentro del flujo general de una CNN, la primera operación clave es la convolución, encargada de extraer patrones locales de la imagen. Un filtro o *kernel* se desplaza por la entrada y, en cada posición, realiza una combinación ponderada de los píxeles que quedan bajo él. Gracias a esta operación, la red puede detectar elementos como bordes, esquinas o texturas.

De forma general, dada una imagen de entrada I y un filtro K de tamaño $m \times m$, la convolución se define como:

$$(I * K)(i, j) = \sum_{u=0}^{m-1} \sum_{v=0}^{m-1} I(i + u, j + v) \cdot K(u, v)$$

Esta expresión indica que:

- El filtro K se coloca sobre la imagen en la posición (i, j) .
- Cada elemento del filtro multiplica al píxel correspondiente de la imagen.
- Todos los productos se suman para generar un único valor en la salida.

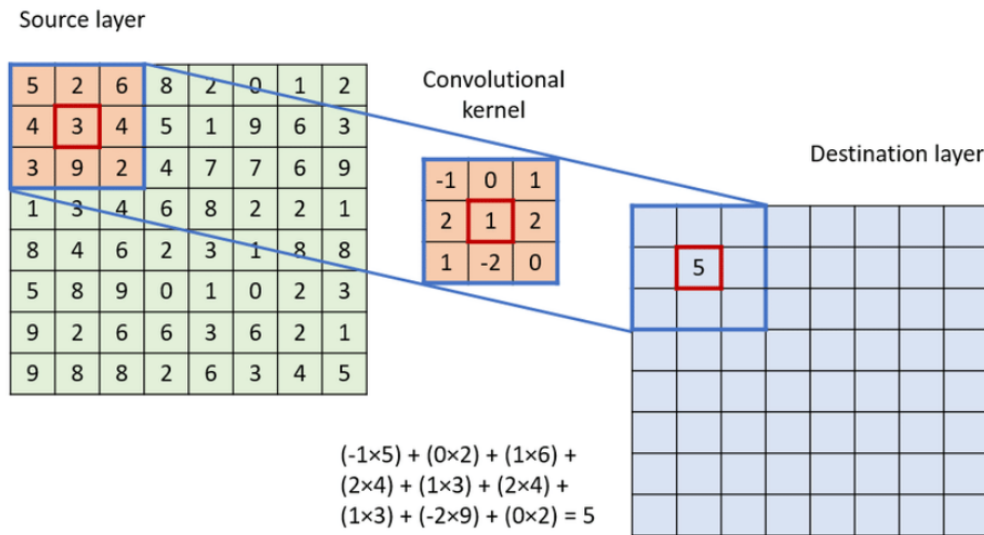


Figura 3.8: Ejemplo del proceso de convolución.

Ejemplo conceptual. Si la imagen es una matriz de 5×5 y el filtro es de 3×3 , en cada posición se multiplican los 9 píxeles de la región cubierta por el filtro y se suman los resultados. Al desplazar el filtro por la imagen, se genera un nuevo mapa donde cada píxel representa la respuesta del filtro en esa ubicación.

Es importante precisar que, aunque la operación de convolución se define matemáticamente de forma bidimensional, en el contexto de este proyecto las imágenes de entrada se procesan en el espacio de color RGB (Red, Green,

Blue). Esto implica que la entrada no es una única matriz, sino un tensor de orden tres con dimensiones $H \times W \times 3$.

En consecuencia, los kernels de la primera capa de la arquitectura ResNet-50 [11], que es la utilizada en este TFG, no son planos, sino que poseen una profundidad equivalente al número de canales de la imagen (3 en este caso). Durante la operación, el kernel desliza un volumen de $k \times k \times 3$ sobre la imagen, calculando el producto escalar de sus pesos con los valores de los tres canales simultáneamente. Los resultados de estos tres canales se suman para generar un único valor escalar en el mapa de características resultante. Este proceso permite que la red capture no solo formas y bordes, sino también patrones cromáticos específicos —como el tono marrón rojizo característico de la Varroa Destructor— desde las primeras etapas del aprendizaje.

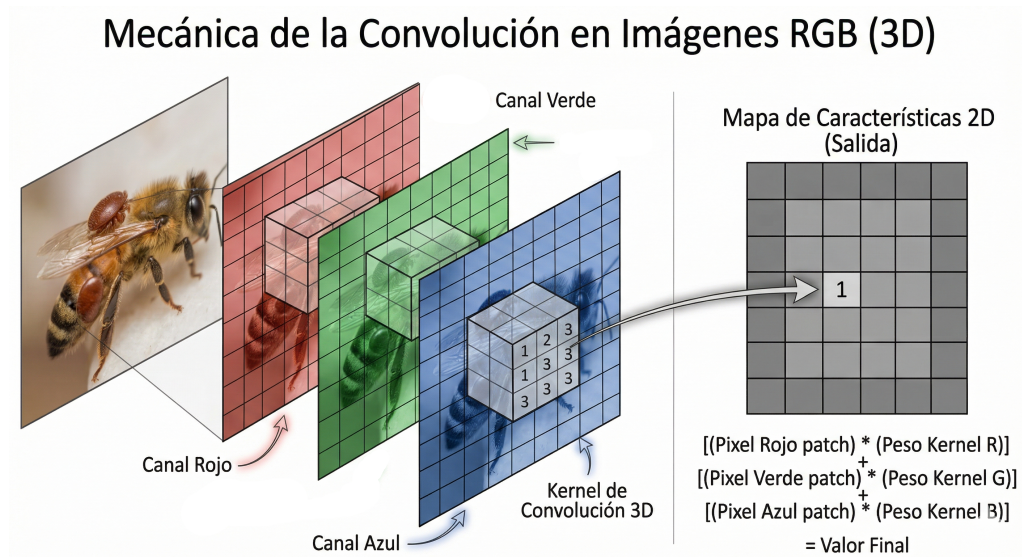


Figura 3.9: Visualización de la convolución en una imagen RGB.

Stride. El *stride* determina cuántos píxeles avanza el filtro en cada paso.

- Un stride de 1 analiza la imagen con mayor detalle.
- Un stride mayor reduce el tamaño del mapa resultante y acelera el cálculo.

Padding. En ocasiones, la imagen se amplía artificialmente añadiendo ceros en los bordes (*padding*). Esto permite controlar el tamaño de la salida y evitar la pérdida de información en los bordes.

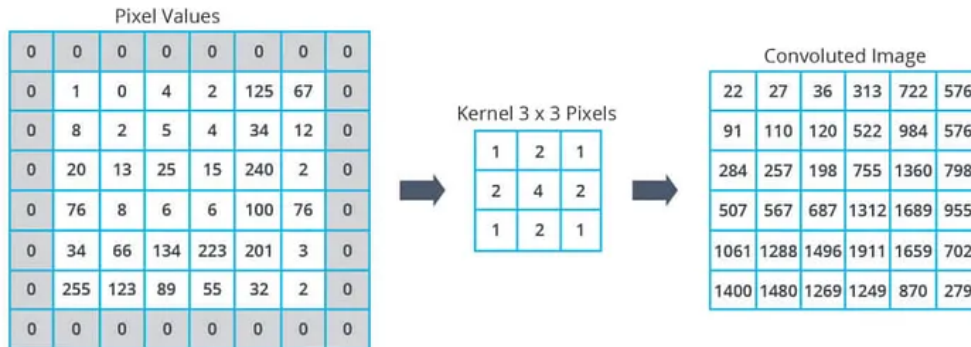


Figura 3.10: Ejemplo de uso del padding.

Mapas de características. Una capa convolucional no utiliza un único filtro, sino muchos. Cada filtro aprende a detectar un tipo de patrón, por lo que su aplicación produce un mapa de características distinto. La salida de una capa convolucional suele estar formada por decenas o cientos de mapas, uno por cada filtro empleado.

Pooling

Tras la extracción inicial de características mediante convoluciones, muchas arquitecturas emplean operaciones de pooling para reducir la resolución espacial de los mapas de características. Esta reducción permite disminuir el coste computacional, reforzar la invarianza a pequeñas variaciones en la imagen y conservar únicamente la información más representativa.

El pooling se aplica mediante una ventana que recorre el mapa de características y resume la información contenida en cada región mediante una operación simple. Las dos variantes más habituales son las siguientes.

Max Pooling. Selecciona el valor máximo dentro de la ventana. Es especialmente útil para resaltar la activación más fuerte en cada región y se emplea frecuentemente en redes destinadas a tareas de visión.

$$\text{MaxPooling}(R) = \max_{(i,j) \in R} x_{ij}$$

Average Pooling. Calcula el valor promedio de los elementos de la ventana. Produce representaciones más suaves, aunque es menos efectivo para conservar detalles discriminativos.

$$\text{AvgPooling}(R) = \frac{1}{|R|} \sum_{(i,j) \in R} x_{ij}$$

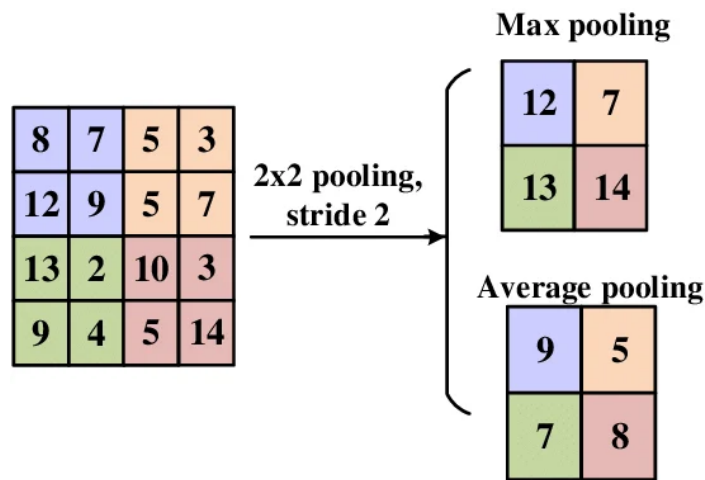


Figura 3.11: Uso de Max Pooling y Average Pooling.

Ejemplo conceptual. Si se aplica una ventana de 2×2 con *stride* 2 sobre un mapa de características, cada grupo de cuatro valores se convierte en un único resultado. De este modo, un mapa de tamaño 8×8 se transforma en uno de 4×4 , reduciendo su tamaño a una cuarta parte.

Aplanado (Flatten) y capas densas

Tras la etapa de extracción y reducción espacial de características mediante convolución y pooling, muchas arquitecturas clásicas de CNN transforman los mapas de características resultantes en un vector unidimensional mediante la operación conocida como *flatten*. Esta transformación permite conectar la parte convolucional con una serie de capas densas encargadas de producir la predicción final.

Aplanado o *flatten*. Los mapas de características generados por las capas previas suelen tener la forma $H \times W \times C$, donde H y W son las dimensiones espaciales y C es el número de canales. El proceso de aplanado reorganiza estos valores en un único vector de tamaño $H \cdot W \cdot C$. Con ello, la red pierde la estructura espacial pero gana una representación compacta apta para operar con capas completamente conectadas.

Capas densas. Una vez obtenido el vector, este se procesa mediante una o varias capas densas. Estas capas combinan la información aprendida durante las etapas anteriores para generar la salida final, ya sea una clasificación, una probabilidad o un conjunto reducido de valores numéricos.

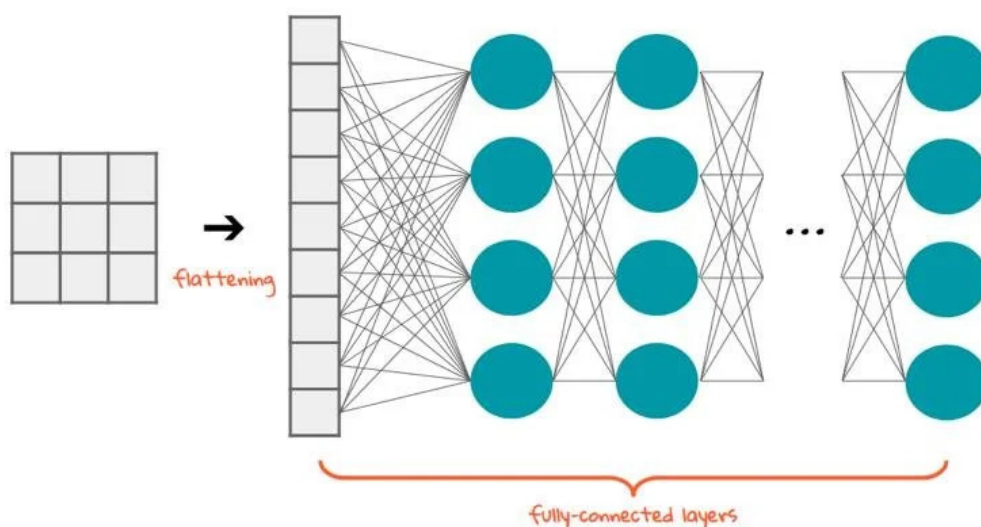


Figura 3.12: Ejemplo simple de flatten.

Por qué una CNN clásica no es suficiente para la detección de objetos

Aunque las CNN tradicionales son muy eficaces en tareas de clasificación, presentan limitaciones importantes cuando el objetivo es localizar objetos dentro de una imagen. La detección requiere identificar no solo qué aparece en la imagen, sino también en qué región se encuentra cada instancia.

Pérdida de información espacial. Al final de una CNN clásica la operación *flatten*, elimina la estructura espacial presente en la imagen. Como

consecuencia, la red ya no puede distinguir la ubicación de los patrones detectados, por lo que solo es capaz de producir una única predicción global.

Variación de escala. En una imagen pueden coexistir objetos de tamaños muy diferentes. Las CNN convencionales procesan la entrada a una única resolución interna, por lo que tienen dificultades para reconocer simultáneamente objetos pequeños y grandes. Esta limitación afecta especialmente a tareas donde los objetos de interés ocupan una fracción reducida de la imagen.

Resumen. Estas limitaciones hacen que una CNN clásica no pueda utilizarse directamente para la detección de objetos. En los capítulos siguientes se presentan las arquitecturas diseñadas específicamente para resolver este tipo de tareas.

3.3. Detectores basados en R-CNN

Los modelos de la familia R-CNN (*Region-based Convolutional Neural Networks*) [6] surgieron como respuesta a una limitación fundamental de las CNN clásicas: su incapacidad para localizar objetos dentro de la imagen. Mientras que una CNN estándar solo produce una predicción global, las arquitecturas R-CNN incorporan mecanismos para proponer regiones candidatas y analizarlas de forma independiente, permitiendo determinar qué objeto aparece y en qué lugar.

A lo largo del tiempo estos modelos han evolucionado mediante mejoras significativas en eficiencia y precisión. Las tres variantes principales —R-CNN, Fast R-CNN y Faster R-CNN— conforman un desarrollo progresivo que culmina en la arquitectura utilizada en este proyecto.

R-CNN: Región basada en una red convolucional

El modelo R-CNN original introduce la idea clave de dividir el problema de detección en dos etapas: localizar posibles regiones de interés (RoI, por sus siglas en inglés) y clasificarlas.

Funcionamiento general.

1. Dada una imagen de entrada, se generan aproximadamente 2000 regiones propuestas utilizando el algoritmo *Selective Search* [29].

2. Cada región propuesta se recorta y se redimensiona.
3. Cada región se pasa por una CNN para extraer un vector de características.
4. Un clasificador externo determina si la región contiene un objeto y de qué clase.
5. Un regresor lineal ajusta las coordenadas de la caja para mejorar la precisión de la localización.

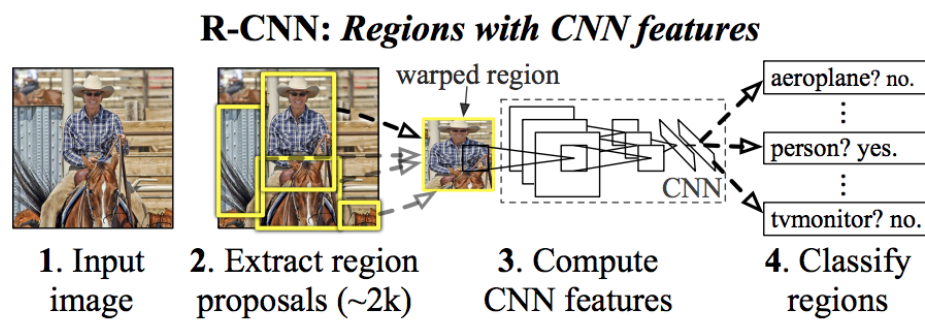


Figura 3.13: Ejemplo de funcionamiento de una R-CNN.

Selective Search. Es un algoritmo que agrupa regiones similares en la imagen utilizando técnicas jerárquicas de segmentación. Produce múltiples cajas candidatas sin necesidad de aprendizaje previo, pero es computacionalmente costoso.

Limitaciones.

- Cada región debe procesarse con la CNN por separado lo que lo hace extremadamente lento.
- El entrenamiento requiere múltiples etapas desconectadas.
- No es adecuado para aplicaciones en tiempo real.

A pesar de estas limitaciones, R-CNN introdujo la idea fundamental que daría lugar a los detectores modernos.

Fast R-CNN

La arquitectura Fast R-CNN [22] surge como respuesta a los elevados costes computacionales de su predecesor. La mejora fundamental de este modelo radica en la optimización del procesamiento espacial: en lugar de extraer y procesar cada región propuesta de forma aislada, el modelo realiza un único pase de la imagen completa a través de la red convolucional. Posteriormente, las regiones de interés se extraen directamente del mapa de características global resultante.

El flujo de procesamiento del modelo se estructura en las siguientes fases:

1. **Extracción global de características:** La imagen original se procesa íntegramente a través de una CNN profunda para generar un único mapa de características multidimensional.
2. **Proyección de regiones:** Las propuestas generadas por algoritmos externos como *Selective Search* se proyectan espacialmente sobre este mapa de características global.
3. **Capa RoI Pooling:** Dado que las cajas propuestas poseen tamaños y relaciones de aspecto variables, esta capa extrae el bloque de características de cada RoI y lo redimensiona mediante operaciones estadísticas como el *max pooling* para generar un tensor de salida de dimensiones fijas. Esta estandarización es un paso algorítmico crucial para poder alimentar las capas densas posteriores.
4. **Procesamiento bicéfalo:** Cada tensor de tamaño fijo ingresa en una red neuronal densa que se divide en dos ramas: un clasificador y un regresor.

Salidas de la red (clasificación y regresión) Para cada región evaluada, la red emite un doble veredicto:

- **Clasificador Softmax:** Determina la probabilidad de que el objeto contenido en la región pertenezca a una clase específica (*Varroa* o Fondo). Dado un vector de puntuaciones de salida z , la función Softmax normaliza los valores en una distribución de probabilidad:

$$\text{Softmax}(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

- **Regresor de cajas delimitadoras:** Predice un vector de ajustes espaciales ($\Delta x, \Delta y, \Delta w, \Delta h$) cuyo objetivo es refinar la posición, el ancho y el alto de la caja propuesta original, ajustándola con precisión a los límites reales del objeto detectado.

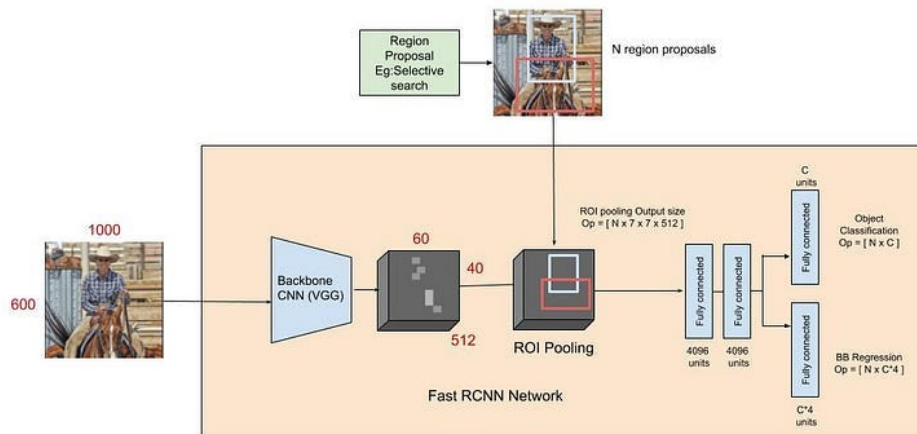


Figura 3.14: Flujo de funcionamiento de un Fast R-CNN.

Ventajas y limitación principal La adopción de Fast R-CNN supone un salto cualitativo en la velocidad de inferencia, ya que el pesado cálculo convolucional se comparte y se ejecuta una sola vez por imagen. Además, permite un entrenamiento más unificado de la arquitectura.

No obstante, el modelo presenta una limitación crítica: sigue dependiendo del algoritmo clásico *Selective Search* [29] para la generación de las propuestas de región. Este proceso, que se ejecuta en la CPU de forma independiente a la red neuronal, supone un severo cuello de botella computacional. La necesidad de eliminar esta dependencia externa es lo que conduce directamente a la siguiente evolución arquitectónica: **Faster R-CNN**.

Faster R-CNN

La arquitectura Faster R-CNN [22], usada en este proyecto, representa un punto de inflexión en la detección de objetos al eliminar el principal cuello de botella de sus predecesores: la dependencia de algoritmos externos para la propuesta de regiones. Para lograrlo, introduce la *Region Proposal*

Network (RPN), una red neuronal completamente convolucional integrada en el propio modelo que asume la tarea de generar las regiones de interés (RoI). Esta integración permite que todo el sistema se entrene de forma unificada, mejorando drásticamente tanto la velocidad de inferencia como la precisión.

El procesamiento en Faster R-CNN se produce de la siguiente manera:

1. **Extracción de características (*Backbone*):** La imagen completa atraviesa una red convolucional profunda que genera un mapa de características global.
2. **Generación de propuestas (RPN):** La red RPN escanea este mapa y propone miles de regiones candidatas.
3. **Filtrado mediante NMS:** Se aplica el algoritmo *Non-Maximum Suppression* para descartar propuestas redundantes.
4. **Clasificación y regresión:** Las RoI seleccionadas se estandarizan (vía *RoI Pooling* o *RoI Align*) y se procesan mediante capas densas para finalmente dar: la probabilidad de clase y el ajuste de las coordenadas de la caja.

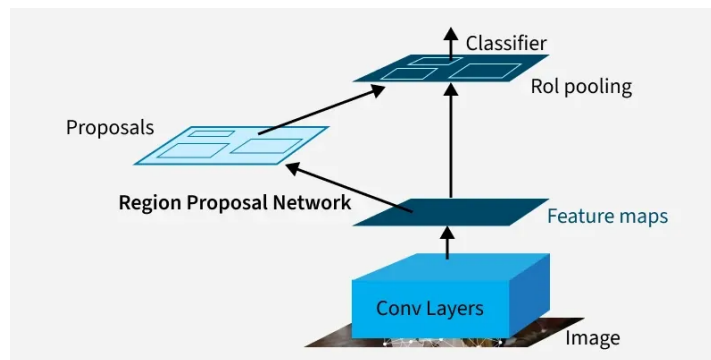


Figura 3.15: Simplificación de un Faster R-CNN.

Region Proposal Network (RPN) y Anchors

En lugar de buscar agrupaciones de píxeles a ciegas como hacía *Selective Search* [29], la RPN [22] se desliza sobre el mapa de características global evaluando múltiples cajas de referencia predefinidas, denominadas *anchors*.

En cada posición espacial, la RPN analiza *anchors* de distintas escalas (tamaños) y relaciones de aspecto (proporciones).

Por cada *anchor*, la RPN emite dos predicciones fundamentales:

- **Puntuación de objeto (*Objectness score*):** La probabilidad de que la caja contenga un objeto de interés frente a ser simplemente fondo.
- **Ajustes espaciales:** Los deltas numéricos necesarios para que el *anchor* original se adapte con mayor precisión a los límites reales del objeto detectado.

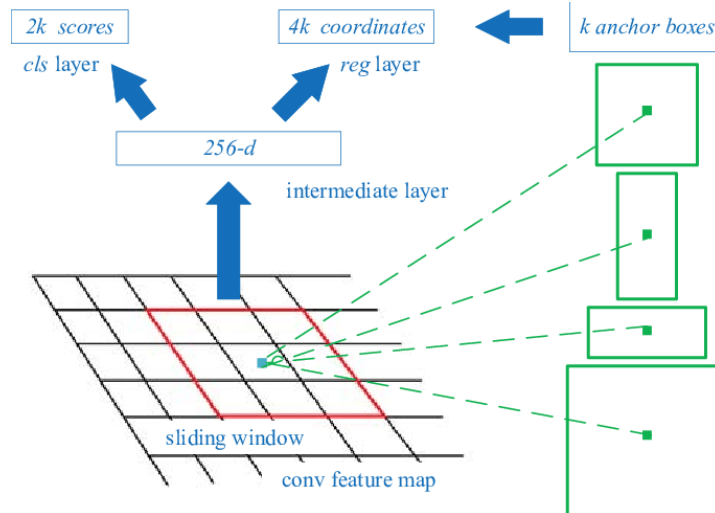


Figura 3.16: Ejemplo de un RPN seccionando una imagen.

Filtrado de propuestas: Non-Maximum Suppression (NMS)

La evaluación exhaustiva de *anchors* genera inevitablemente una gran cantidad de cajas solapadas sobre un mismo objeto. Para resolver esta redundancia, se aplica el algoritmo *Non-Maximum Suppression* (NMS). Este algoritmo ordena las cajas propuestas según su *objectness score*, selecciona la de mayor puntuación y elimina todas aquellas que presenten un solapamiento excesivo con la ganadora, medido a través del índice *Intersection over Union* (IoU) [23]. De este modo, la red transmite únicamente las propuestas más informativas y precisas a las capas finales.

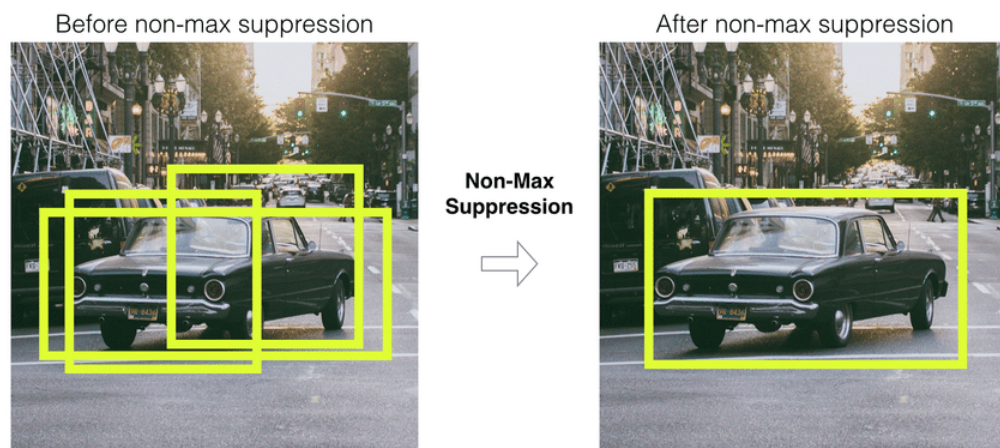


Figura 3.17: Ejemplo de uso de NMS.

Arquitectura	Procesamiento Espacial	Propuestas de Región	Velocidad inferencia	Rol en la Detección
RNA	Vector 1D (pierde topología espacial).	No aplica.	Lenta.	Obsoleta para procesar imágenes completas.
CNN Clásica	Procesa la imagen por zonas manteniendo su forma.	No aplica.	Rápida.	Clasifica la imagen global, pero no localiza.
R-CNN	CNN ~ 2000 veces (una por cada región).	<i>Selective Search</i> .	Muy lenta ($\sim 40s$).	Primer modelo funcional, inviable para uso real.
Fast R-CNN	CNN 1 sola vez (genera 1 mapa global).	<i>Selective Search</i> .	Lenta ($\sim 2s$).	Mejora drástica, pero lastrada por el buscador de RoIs.
Faster R-CNN	CNN 1 sola vez (genera 1 mapa global).	RPN.	Muy rápida ($\sim 0.2s$).	Integra todo el proceso de detección en una sola red neuronal.

Tabla 3.1: Comparativa y evolución de las arquitecturas de detección de objetos.

3.4. Métricas de evaluación

Para evaluar el rendimiento de un sistema de detección de objetos no es suficiente con conocer si el modelo acierta o falla de forma global. En este tipo de problemas resulta fundamental medir tanto la capacidad del modelo para localizar correctamente los objetos como su habilidad para clasificarlos de forma precisa.

A continuación se muestran algunas de las métricas más habituales en detección de objetos: *Intersection over Union* (IoU), Precision, Recall, F1-score y *mean Average Precision* (mAP). Estas métricas permiten analizar el comportamiento del modelo desde distintos puntos de vista y comparar su rendimiento de forma objetiva.

Intersection over Union (IoU)

La métrica *Intersection over Union* (IoU) [23] se utiliza para cuantificar el grado de solapamiento entre una caja delimitadora predicha por el modelo y la caja real anotada en el conjunto de datos.

Formalmente, IoU se define como:

$$IoU = \frac{\text{Área de intersección}}{\text{Área de la unión}}$$

El valor de IoU oscila entre 0 y 1:

- IoU = 0 indica que no existe solapamiento entre las cajas.
- IoU = 1 indica que ambas cajas coinciden exactamente.

En detección de objetos se fija normalmente un umbral mínimo de IoU (por ejemplo, 0.5) para considerar que una detección es correcta. Si el solapamiento entre la caja predicha y la real supera dicho umbral, la detección se clasifica como positivo.

Precisión

La precisión mide la proporción de detecciones realizadas por el modelo que son correctas. Es decir, indica cuántas de las varroas detectadas realmente corresponden a una varroa existente en la imagen.

Se define como:

$$\text{Precisión} = \frac{TP}{TP + FP}$$

donde:

- TP (*True Positives*) son las detecciones correctas.
- FP (*False Positives*) son las detecciones incorrectas.

Una alta precisión indica que el modelo comete pocos falsos positivos. En el contexto de este proyecto, una baja precisión implicaría que el sistema detecta varroas donde realmente no las hay.

Recall

El *Recall* mide la capacidad del modelo para detectar todos los objetos presentes en la imagen. Indica qué proporción de las varroas reales han sido detectadas correctamente.

Se define como:

$$\text{Recall} = \frac{TP}{TP + FN}$$

donde:

- FN (*False Negatives*) son las varroas que el modelo no ha detectado.

Un valor alto de recall significa que el modelo pasa por alto pocas varroas. En este proyecto, un recall bajo implicaría que el sistema no detecta una parte significativa de los parásitos presentes en las imágenes.

F1-score

La métrica F1-score combina la precisión y el recall en un único valor, proporcionando una medida equilibrada del rendimiento del modelo.

Se define como la media armónica entre precisión y recall:

$$F1 = 2 \cdot \frac{\text{Precisión} \cdot \text{Recall}}{\text{Precisión} + \text{Recall}}$$

El F1-score resulta especialmente útil cuando existe un compromiso entre precisión y recall, ya que penaliza situaciones en las que una de las dos métricas es alta y la otra muy baja.

Mean Average Precision (mAP@0.5)

La métrica *mean Average Precision* (mAP) [5] es el estándar más utilizado para evaluar detectores de objetos. Tiene en cuenta tanto la precisión como el recall para distintos umbrales de confianza del modelo.

Para una clase determinada, se construye una curva Precisión–Recall variando el umbral de confianza de las detecciones. El *Average Precision* (AP) se obtiene como el área bajo dicha curva. Cuando se dispone de varias clases, el valor final mAP se calcula como la media del AP de todas ellas.

En este proyecto se utiliza la métrica mAP@0.5, que indica que:

- Una detección se considera correcta si el IoU es mayor o igual a 0.5;
- El valor mAP resume el comportamiento global del modelo para este criterio de solapamiento.

Dado que el problema abordado consiste en la detección de una única clase, el valor de mAP coincide con el AP de dicha clase. Esta métrica permite comparar de forma objetiva el rendimiento del modelo y evaluar su capacidad global de detección.

4. Técnicas y herramientas

En este capítulo se describen las herramientas tecnológicas, la arquitectura del modelo seleccionado y los procedimientos de optimización implementados para la detección automática de la *Varroa Destructor*.

4.1. Entorno de desarrollo y tecnologías

- **Lenguajes de programación:** Se ha utilizado **Java** [8] para el desarrollo del servidor web principal debido a su robustez, y **Python** [30] para el ecosistema de Inteligencia Artificial debido a su enorme facilidad de integración y su posición como estándar absoluto en el desarrollo de modelos de aprendizaje profundo.
- **Framework de IA: PyTorch** [20] ha sido el entorno empleado para definir y entrenar la red neuronal. Se ha elegido porque permite construir el modelo de forma dinámica, lo que facilita enormemente encontrar y corregir errores durante la programación.
- **Procesamiento de imagen:** Se han utilizado bibliotecas especializadas como OpenCV [2] para las operaciones de manipulación matricial de las imágenes y Albumentations [3] como motor principal para la generación de aumentos de datos (Data Augmentation) [27], seleccionada por su eficiencia y su capacidad para aplicar transformaciones complejas manteniendo la coherencia de las cajas delimitadoras.
- **Hardware:** El modelo se ha entrenado en hardware local utilizando una tarjeta gráfica **NVIDIA GeForce RTX 4060 Ti**. Dispone de suficiente memoria y capacidad de procesamiento paralelo para realizar cálculos simultáneos, reduciendo los tiempos de entrenamiento.

4.2. Arquitectura del modelo

La elección de la arquitectura responde a la necesidad de obtener una alta precisión espacial en la identificación de objetos de dimensiones reducidas.

- **Modelo principal (Faster R-CNN):** [22] Cuyo funcionamiento se ha detallado en el apartado teórico. Se ha elegido esta arquitectura porque a diferencia de modelos ultrarrápidos de una sola fase como YOLO esta pérdida de velocidad para realizar sus dos fases resulta fundamental para no perder precisión espacial al intentar detectar objetos diminutos como la varroa.
- **Backbone (ResNet-50):** [11] Actúa como el motor extractor de características visuales. Se ha seleccionado la variante de 50 capas para superar el problema del desvanecimiento del gradiente (vanishing gradient problem). De forma intuitiva, cuando una red es muy profunda, la señal de error debe atravesar tantas capas durante el proceso de backpropagation que la información original se va distorsionando y perdiendo fuerza antes de llegar a las capas iniciales. Para solucionar esto, ResNet-50 introduce las conexiones residuales (o skip connections), que permiten a los datos saltarse ciertas capas intermedias para llegar intactos a niveles más profundos; estas conexiones son, de hecho, las que dan nombre a la arquitectura (Residual Networks). Matemáticamente, si el objetivo de un bloque es aprender una función $H(x)$, la conexión residual suma la entrada original directamente a la salida, redefiniendo la operación como $F(x)+x$. Este atajo matemático permite que el modelo alcance la profundidad necesaria para extraer los detalles morfológicos del parásito, garantizando que el aprendizaje fluya por toda la red sin saturarse.

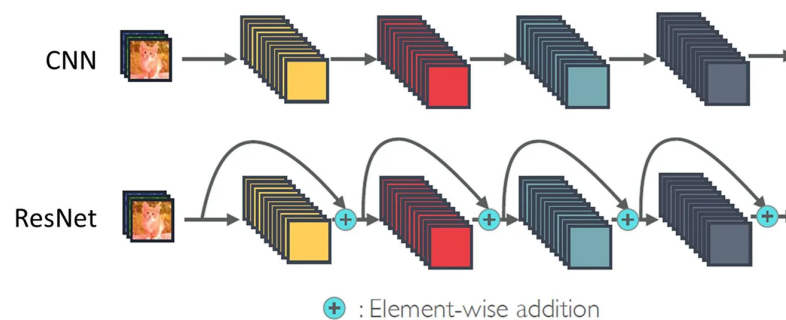


Figura 4.1: Comparativa de una CNN con ResNet.

- **Adaptación multiescala (FPN):** La *Feature Pyramid Network* es una extensión arquitectónica diseñada expresamente para resolver el problema de la variabilidad de escalas. Las redes convolucionales tradicionales pierden resolución espacial a medida que ganan comprensión semántica. FPN soluciona esto construyendo una arquitectura de dos vías: proyecta las características fuertes de las capas más profundas hacia las capas más superficiales mediante conexiones laterales. Para este proyecto, su integración es crítica, ya que garantiza que las varroas, al ser objetos diminutos, no desaparezcan en los mapas de baja resolución de las últimas capas al fusionar la información de contexto con los detalles de alta resolución.

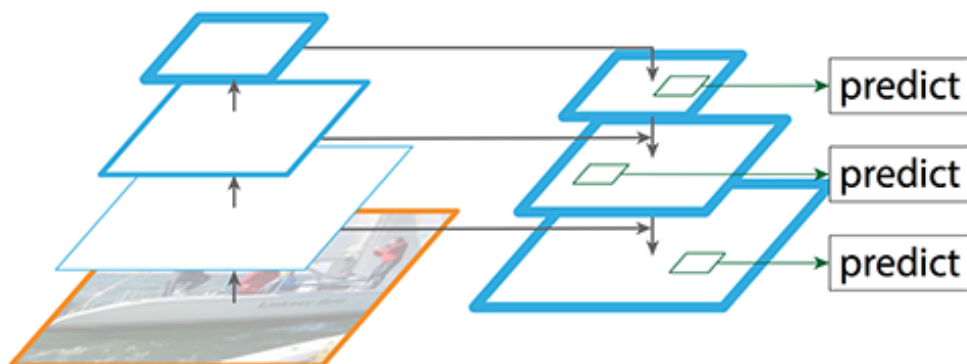


Figura 4.2: Vista simplificada del funcionamiento de FPN.

4.3. Preparación del dataset

- **Formato de datos:** Se ha trabajado con un corpus de **429 imágenes** originales. La técnica empleada para enseñar al modelo dónde está el parásito ha sido el formato de cajas delimitadoras (*bounding boxes*). Se han procesado los metadatos a través de archivos estándar XML (PASCAL VOC) [5] que contienen las coordenadas exactas.
- **División del conjunto de datos:** Se realizó una división del conjunto de imágenes aplicando el estándar metodológico para evitar el sobreajuste: 70 % de las imágenes para entrenamiento, 20 % para validación y 10 % para el conjunto de test final.

- **Aumento de datos (*Data Augmentation*):** [27] Para paliar la escasez de imágenes originales necesarias en un entrenamiento profundo, se aumentó el tamaño del dataset utilizando *Albumentations* [3]. Se aplicaron transformaciones de nitidez, rotación e iluminación forzando al modelo a aprender características invariantes y haciéndolo robusto frente a las diversas condiciones del apiario.

Conjunto (Split)	Volumen final de imágenes
Entrenamiento (Train)	1014
Validación (Val)	289
Prueba (Test)	146
TOTAL	1449

Tabla 4.1: Distribución final del conjunto de datos tras la aumentación (partiendo de 429 imágenes originales y 1020 generadas sintéticamente).

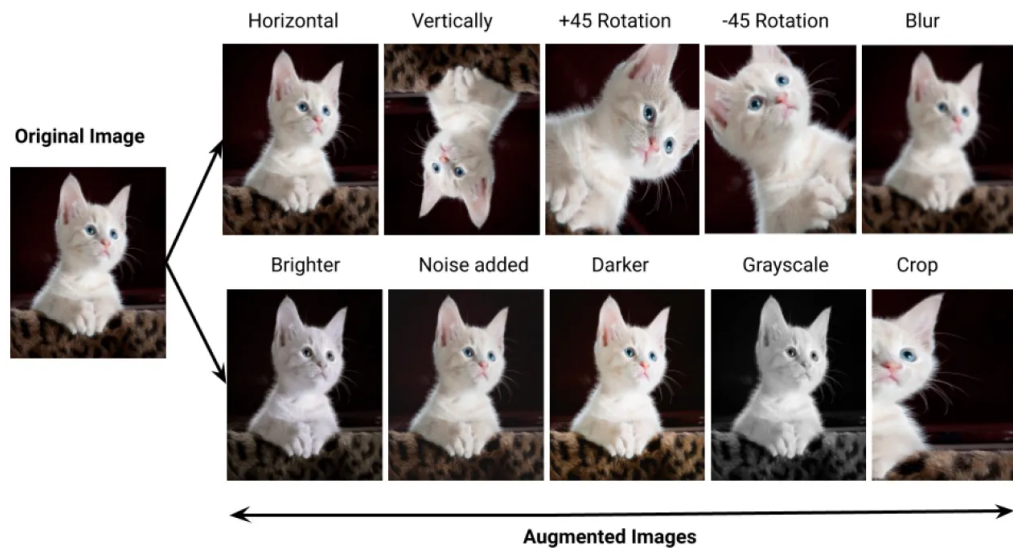


Figura 4.3: Ejemplo de resultado del uso de la biblioteca albumentation.

4.4. Proceso de entrenamiento y optimización

- **Aprendizaje por transferencia (*Transfer Learning*):** [19] El entrenamiento de redes profundas requiere volúmenes masivos de datos. Para superar la limitación de 429 imágenes, se ha empleado el aprendizaje por transferencia. En lugar de inicializar los parámetros aleatoriamente, se ha cargado el modelo con pesos matemáticos pre-entrenados en el macro-dataset **MS COCO** [15]. Gracias a esto, las primeras capas ya poseen la capacidad de extraer características visuales universales. El entrenamiento se limita a un proceso de ajuste fino (*fine-tuning*), especializando las últimas capas para que reconozcan cómo esos patrones conforman el parásito, acelerando radicalmente la convergencia.

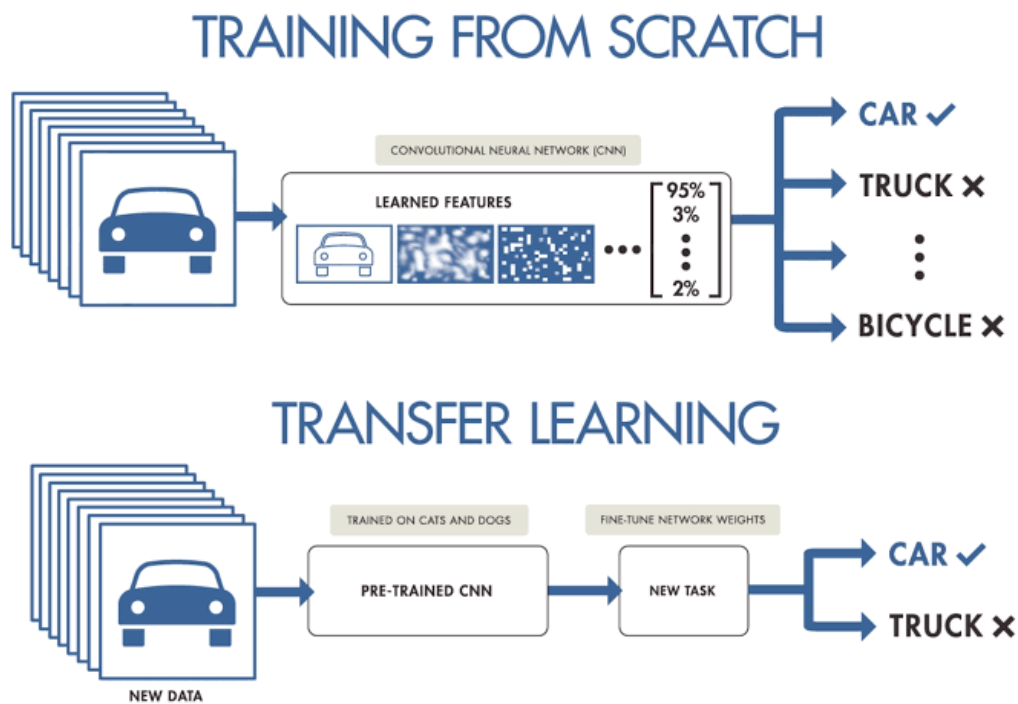


Figura 4.4: Comparativa entrenamiento normal y transfer learning.

- **Optimizador:** El algoritmo **SGD** (Descenso de Gradiente Estocástico) con *momentum* (0.9) [24, 21] ha sido el elegido, cuyo concepto se explica en el apartado teórico. Se seleccionó porque, en problemas de detección

geométrica compleja, tiende a conseguir un ajuste final mucho más estable.

- **Hiperparámetros de entrenamiento:** Para la optimización matemática del modelo se establecieron los siguientes parámetros y técnicas de regularización:
 - **Épocas y tamaño de lote:** El entrenamiento se ejecutó durante un total de **20 épocas**, procesando la información con un tamaño de lote (*batch size*) de **2 imágenes** por iteración.
 - **Tasa de aprendizaje (*Learning Rate*) y Planificador (*StepLR*):** Se estableció un valor inicial de **0.005**. Se aplicó un planificador escalonado (*StepLR*) que reduce esta tasa multiplicándola por un factor de 0.1 cada 3 épocas. Esta técnica permite realizar actualizaciones de gran magnitud en las fases iniciales y transicionar a ajustes mas pequeños en las fases finales, evitando oscilaciones bruscas cerca del óptimo de la función de error.

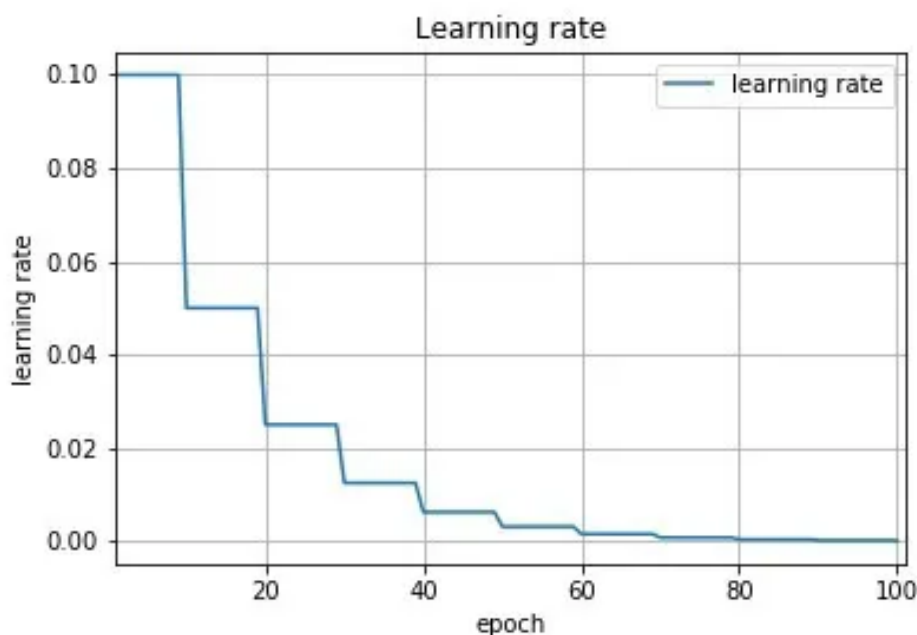


Figura 4.5: Ejemplo de uso de StepLR.

- **Decaimiento de pesos (*Weight Decay*):** Establecido en **0.0005**. Esta técnica aplica una regularización matemática (conocida como norma L2) que añade a la función de pérdida una

penalización proporcional al cuadrado de la magnitud de los pesos. Su finalidad es evitar que parámetros individuales crezcan en exceso y dominen el proceso de predicción, obligando a la red neuronal a distribuir el aprendizaje de forma equilibrada entre todas sus conexiones. Esto resulta muy útil para mitigar el riesgo de sobreajuste y mejorar la generalización del modelo sobre un conjunto de datos reducido.

4.5. Despliegue e interfaz de usuario

Para dotar al proyecto de aplicabilidad real y facilitar su uso a usuarios sin conocimientos de programación (apicultores), se ha desarrollado una aplicación web.

- **Backend y enrutamiento:** Se ha implementado el servidor web principal utilizando **Spring Boot** [33, 31] en Java, el cual actúa como controlador central gestionando las peticiones HTTP, el control de accesos de usuarios y la persistencia en la base de datos **H2 embebida** [10]. Este componente interactúa de forma desacoplada con un micro-servicio secundario en **Flask** [9] (Python), encargado en exclusiva de recibir las imágenes y orquestar la inferencia llamando a PyTorch.
- **Frontend y motor de plantillas:** La interfaz gráfica del cliente se ha estructurado mediante HTML5 [32] y CSS3 (apoyado en Bootstrap 5 [18]). Para la integración de datos dinámicos, se ha integrado el motor de plantillas **Thymeleaf** [28], el cual permite renderizar en tiempo real el histórico de análisis, los datos de usuario y las rutas de las imágenes resultantes dentro de la vista web.
- **Pipeline de inferencia web:** El sistema carga los pesos óptimos del modelo (`weights_varroa_best.pth`). Al recibir una imagen, se realiza la inferencia aplicando un umbral de confianza estricto del 50 % (`conf_thresh=0.5`). Todas las detecciones válidas son utilizadas por OpenCV [2] para dibujar las cajas delimitadoras y las puntuaciones de confianza sobre la imagen original. Finalmente, el servidor devuelve al cliente la imagen procesada junto con el sumatorio total de ácaros detectados.
- **Estrategia de despliegue:** El sistema utiliza un enfoque diseñado para maximizar su portabilidad local sin depender de software de terceros. La capa de persistencia utiliza la base de datos **H2** [10] de

forma embebida. De este modo, las capas de lógica y de datos se ejecutan de manera nativa sobre el sistema host, coordinadas de forma automática mediante un script de ejecución (`iniciar_todo.bat`).

5. Aspectos relevantes del desarrollo del proyecto

En este apartado se recogen los aspectos más relevantes relacionados con el desarrollo práctico del proyecto. Se detallan las decisiones arquitectónicas tomadas para la construcción del software, los retos técnicos afrontados durante la implementación y, finalmente, se exponen los resultados obtenidos por el modelo de Inteligencia Artificial.

5.1. Metodología y ciclo de vida

Dada la naturaleza del proyecto, centrado en la extracción de conocimiento a partir de imágenes, no se ha seguido un modelo de ingeniería de software tradicional. En su lugar, se ha optado por la metodología **CRISP-DM** (*Cross-Industry Standard Process for Data Mining*) [34], el estándar más extendido en proyectos de ciencia de datos y aprendizaje automático.

CRISP-DM propone un ciclo de vida iterativo que permite retroceder a fases previas basándose en los descubrimientos realizados. Para este proyecto, el flujo se ha adaptado en las siguientes etapas fundamentales:

- **Comprensión de los datos:** Análisis del comportamiento del ácaro varroa y evaluación de las limitaciones del *dataset* inicial (baja resolución y escasez de muestras).
- **Preparación de los datos:** Fase crítica donde se ejecutaron los *scripts* de limpieza y se aplicó *Data Augmentation* [27] mediante la biblioteca Albumentations [3] para robustecer el conjunto de entrenamiento.

- **Modelado:** Configuración y entrenamiento de la arquitectura *Faster R-CNN* [22] utilizando técnicas de *Transfer Learning* [19] sobre pesos pre-entrenados en COCO [15].
- **Evaluación:** Análisis de las métricas de precisión (mAP) y determinación del impacto del desenfoque en la detección.
- **Despliegue:** Integración del modelo final en una arquitectura web funcional híbrida controlada por el servidor principal de **Spring Boot** [31] en comunicación con la API de **Flask** [9].

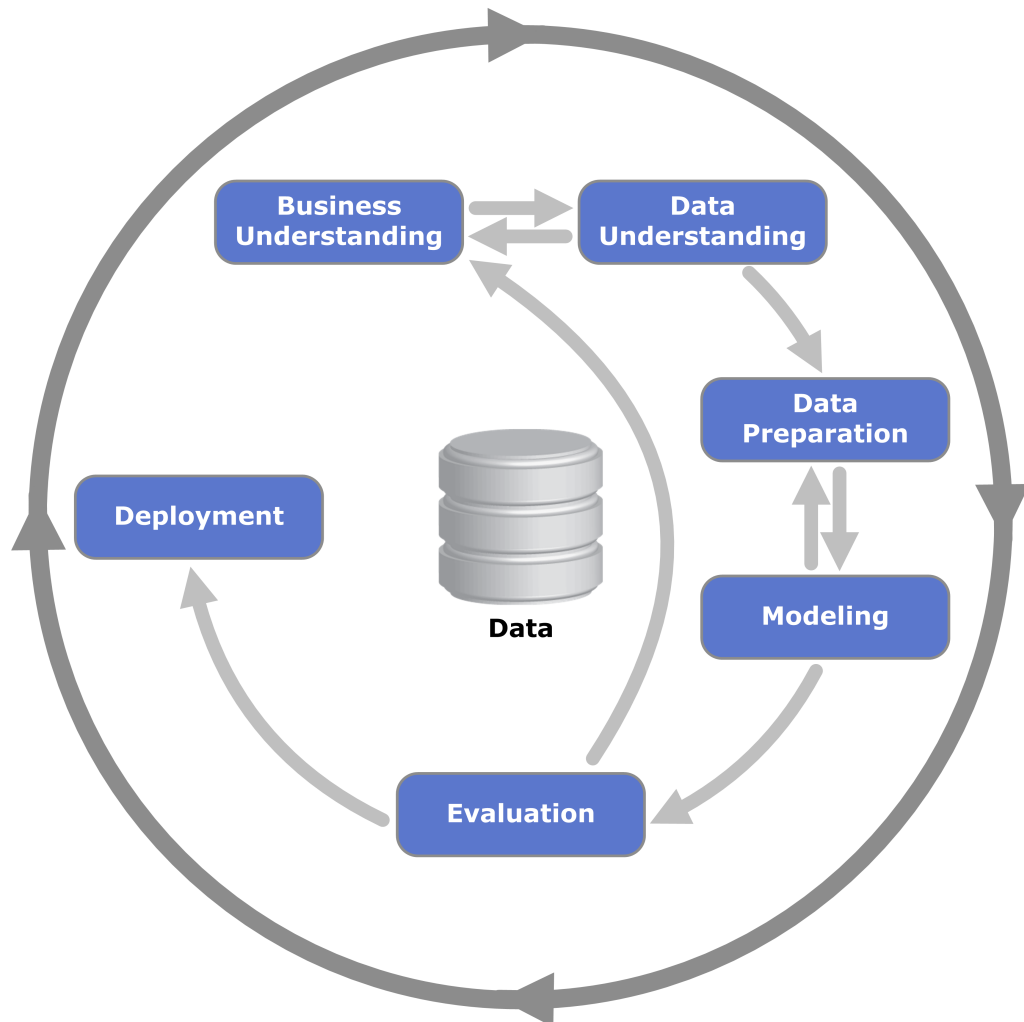


Figura 5.1: Flujo de trabajo de CRISP-DM.

5.2. Arquitectura del sistema y desarrollo

El objetivo principal de la implementación fue construir un sistema funcional que permitiese a un apicultor subir una fotografía y obtener una inferencia en tiempo real.

Desarrollo del backend y modelo en memoria

La arquitectura lógica se compone de un servidor web principal en **Spring Boot** [31] (Java) y una API en **Flask** [9] (Python). La aplicación (`app.py`) actúa como un microservicio que sirve de puente entre el servidor Java y el tensor de cálculo numérico de PyTorch [20].

Para garantizar la eficiencia operativa, el modelo *Faster R-CNN* [22] y sus pesos asociados (`weights_varroa_best.pth`) se cargan en la memoria RAM una única vez durante el arranque del servidor. Esto evita los tiempos muertos de inicialización en cada petición.

Procesamiento e interfaz (*frontend*)

El flujo de ejecución (*pipeline*) se programó para ser transparente al usuario:

- El usuario selecciona una o varias imágenes a través de la interfaz web, las cuales son enviadas al servidor de Java de forma asíncrona.
- El servicio de Python recibe la imagen, aplica transformaciones matriciales y la convierte en un tensor de PyTorch normalizado.
- Una vez que la red neuronal devuelve las coordenadas espaciales de la predicción, se utilizó la biblioteca de visión artificial **OpenCV** [2] (`cv2`) para dibujar las cajas delimitadoras y estampar el porcentaje de confianza (*score*) directamente sobre una copia de la imagen original.
- Finalmente, la imagen procesada se envía de vuelta a Java, se almacena y se sirve a la vista HTML del cliente.

5.3. Retos técnicos y limitaciones

Durante el desarrollo del sistema, la principal barrera técnica encontrada no residió en la arquitectura de red, sino en un cuello de botella clásico en

el ámbito del *Machine Learning*: el límite de calidad intrínseca de los datos de entrada (*Garbage In, Garbage Out*).

El *dataset* original presentaba dos carencias fundamentales:

1. **Escasez volumétrica:** Se solucionó exitosamente mediante la implementación de un motor de *Data Augmentation* [27] (*Albumentations* [3]), generando variantes fotométricas y geométricas.
2. **Deficiencia de resolución y ruido orgánico:** El entorno de evaluación del modelo son los paneles blancos de recuento (suelos sanitarios) extraídos de las colmenas. Aunque el fondo blanco facilita el contraste, el panel suele estar cubierto de ruido orgánico (restos de cera, polen y suciedad general). Al ser la *Varroa* un ácaro de dimensiones milimétricas, cuando la imagen de entrada presenta desenfoque o baja resolución, el parásito pierde sus características morfológicas. Esto provoca que la red neuronal confunda al ácaro con simple suciedad de la colmena y viceversa.

Aunque se implementaron múltiples iteraciones de ajuste de hiperparámetros (*Hyperparameter Tuning*) en el algoritmo de optimización para forzar la convergencia, se determinó que el rendimiento del modelo está directamente condicionado por la capacidad de la cámara que tome las fotografías, siendo indispensable una alta nitidez para discriminar la plaga de los detritos.



Figura 5.2: Imagen del dataset mostrando la falta de resolución.

5.4. Evaluación y resultados del modelo

El rendimiento del modelo se evaluó de forma rigurosa sobre el conjunto de *test*, estableciendo un umbral de Confianza (*Confidence*) mayor al 50 % y un umbral de Intersección sobre Unión (*IoU*) [23] de 0.5 para considerar una detección como verdadera positiva.

Los resultados extraídos del *script* de evaluación (`Evaluacion.py`) sobre un total de 1166 etiquetas reales y 1608 detecciones arrojadas por la red, son los siguientes:

- **Mean Average Precision (mAP@0.5): 0.3234 (32.34 %).** Esta métrica resume la precisión general del modelo detectando objetos de tamaño extremadamente reducido en el dataset actual.
- **Precisión (*precision*): 0.5817.** Indica que, cuando el modelo marca un objeto asegurando que es una Varroa, acierta en un 58.17 % de las ocasiones. Demuestra una capacidad moderada-alta para controlar los falsos positivos.
- **Sensibilidad (*Recall*): 0.3388.** Indica que el modelo es capaz de localizar al 33.88 % del total de Varroas existentes en una colmena. Esta métrica es la más afectada por el desenfoque de las imágenes, generando un alto volumen de falsos negativos por camuflaje.
- **F1-Score: 0.4282.** Esta media armónica refleja el balance final del modelo, demostrando que la arquitectura es funcional y capaz de detectar la plaga, aunque precisa de la captura de imágenes en alta definición (*HD*) en trabajos futuros para alcanzar umbrales comerciales.

6. Trabajos relacionados

En el ámbito de la visión por computador aplicada a la detección de plagas e insectos, se han propuesto numerosos enfoques basados en aprendizaje profundo [13]. A continuación, se resumen algunos trabajos y recursos relacionados que sirven como referencia para este proyecto.

- **HAIS 2023 — Varroa (paper en PDF)**. Trabajo centrado en el análisis automatizado de varroa en colmenas [1], útil como referencia directa del dominio apícola y del tipo de problema abordado en este TFG. Enlace: <https://www.scitepress.org/Papers/2023/123588/123588.pdf>
- **SP-YOLO (MDPI Insects, 2025)**. Propuesta de un modelo multiescala orientado a detección eficiente y en tiempo real de plagas, basado en una arquitectura tipo YOLO. Es un ejemplo representativo de detectores de una etapa aplicados a escenarios agrícolas. Enlace: <https://www.mdpi.com/2075-4450/16/1/102> Aunque arquitecturas recientes como SP-YOLO ofrecen ventajas en velocidad, se ha optado por Faster R-CNN [22] con FPN debido a su mayor robustez probada en la localización de objetos microscópicos (Varroa) en fondos con alta densidad de residuos orgánicos.
- **Comparativa de Faster R-CNN / Mask R-CNN / YOLOv5 (PDF accesible)**. Estudio que aplica y compara varios detectores de objetos modernos en el contexto de detección de plagas, lo que resulta útil para contextualizar la elección de arquitectura en un problema real. Enlace: <https://pdfs.semanticscholar.org/cfe7/ba3910269e81cc7946693bfd4b5f8379a0bb.pdf>

7. Conclusiones y líneas de trabajo futuras

En este capítulo se presenta un balance final del proyecto, analizando el grado de cumplimiento de los objetivos iniciales y proponiendo una hoja de ruta para la evolución del sistema en futuras versiones.

7.1. Conclusiones

Tras completar el ciclo de desarrollo del sistema de detección de *Varroa Destructor*, se pueden extraer las siguientes conclusiones:

- **Cumplimiento de objetivos:** Se ha logrado desarrollar un prototipo funcional capaz de procesar imágenes de bandejas sanitarias y realizar un conteo automatizado de parásitos.
- **Límites ópticos frente a viabilidad algorítmica:** La elección de la arquitectura *Faster R-CNN* [22] con *Feature Pyramid Network* ha demostrado ser la herramienta algorítmica correcta para localizar objetos milimétricos. Sin embargo, los resultados obtenidos (**mAP@0.5 del 32.34%**) demuestran que el sistema se encuentra limitado por el hardware de captura. La resolución nativa y el ruido orgánico de los paneles blancos dificultan la extracción de características, demostrando que en visión por computador la calidad de la inferencia está directamente relacionada con la nitidez de la entrada.
- **Aprendizaje personal:** El proyecto ha permitido consolidar conocimientos avanzados en Python [30], el ecosistema PyTorch [20] y

el despliegue de aplicaciones web, enfrentando retos reales de visión artificial.

7.2. Posibilidades de mejora y líneas futuras

A pesar de haber logrado un sistema funcional, la arquitectura presenta un margen de mejora que define las siguientes líneas de trabajo productivo:

- **Mejora del *hardware* y preprocesamiento adaptativo:** El rendimiento del modelo es altamente sensible a la captura. Como trabajo futuro, se exige la transición hacia cámaras con mejor calidad de imagen combinado con técnicas como filtros de nitidez automáticos para normalizar las imágenes antes de la inferencia.
- **Despliegue en la nube y escalabilidad:** El prototipo actual funciona en un entorno local. Para convertirlo en una herramienta comercial de uso masivo, la siguiente fase sería el despliegue en servidores remotos para permitir el acceso simultáneo de múltiples usuarios.
- **Funcionalidades extra para el apicultor:** Desde el punto de vista del usuario final, el *software* debería expandirse con:
 - *Ampliación y diversidad del dataset:* Recopilar un mayor volumen de imágenes en diferentes estaciones del año y condiciones lumínicas para mejorar la robustez de la red, e incluso entrenar el modelo para identificar de forma conjunta otras amenazas.
 - *Generación de informes:* Un módulo para exportar los resultados de las inferencias en formato PDF, facilitando la toma de decisiones sobre tratamientos veterinarios.

Bibliografía

- [1] Kim Bjerger, Per Kryger Nielsen, Mads V Sepstrup, and F Helsing-Engler. A computer vision system to monitor the infestation level of varroa destructor on honey bees. *Computers and Electronics in Agriculture*, 158:27–38, 2019.
- [2] Gary Bradski. The opencv library. *Dr. Dobbs's Journal of Software Tools*, 25(11):120–123, 2000.
- [3] Alexander Buslaev, Vladimir I Iglovikov, Eugene Khvedchenya, Alex Parinov, Mikhail Druzhinin, and Alexandr A Kalinin. Albumentations: Fast and flexible image augmentations. *Information*, 11(2):125, 2020.
- [4] Vincent Dietemann, F Nazzi, SJ Martin, DL Anderson, B Locke, KS Delaplane, Q Wauquiez, C Tannahill, E Frey, B Ziegelmann, et al. Standard methods for varroa research. *Journal of Apicultural Research*, 52(1):1–54, 2013.
- [5] Mark Everingham, Luc Van Gool, Christopher KI Williams, John Winn, and Andrew Zisserman. The pascal visual object classes (voc) challenge. *International Journal of Computer Vision*, 88(2):303–338, 2010.
- [6] Ross Girshick, Jeff Donahue, Trevor Darrell, and Jitendra Malik. Rich feature hierarchies for accurate object detection and semantic segmentation. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 580–587, 2014.
- [7] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016.

- [8] James Gosling, Bill Joy, Guy Steele, Gilad Bracha, and Alex Buckley. *The Java language specification*. Addison-Wesley Professional, 2014.
- [9] Miguel Grinberg. *Flask Web Development: Developing Web Applications with Python*. O'Reilly Media, Inc., 2018.
- [10] H2 Group. H2 database engine, 2024. Último acceso en mayo de 2026.
- [11] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778, 2016.
- [12] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. Imagenet classification with deep convolutional neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 1097–1105, 2012.
- [13] Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. Deep learning. *Nature*, 521(7553):436–444, 2015.
- [14] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.
- [15] Tsung-Yi Lin, Michael Maire, Serge Barnes Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár, and C Lawrence Zitnick. Microsoft coco: Common objects in context. In *European Conference on Computer Vision (ECCV)*, pages 740–755. Springer, 2014.
- [16] Stephen J Martin. The role of varroa and viral pathogens in the collapse of honeybee colonies: a modelling approach. *Journal of Applied Ecology*, 38(6):1082–1093, 2001.
- [17] Vinod Nair and Geoffrey E Hinton. Rectified linear units improve restricted boltzmann machines. In *Proceedings of the 27th International Conference on Machine Learning (ICML)*, pages 807–814, 2010.
- [18] Mark Otto and Jacob Thornton. Bootstrap: The most popular html, css, and js library in the world, 2024. Último acceso en mayo de 2026.
- [19] Sinno Jialin Pan and Qiang Yang. A survey on transfer learning. *IEEE Transactions on Knowledge and Data Engineering*, 22(10):1345–1359, 2009.

- [20] Adam Paszke, Sam Gross, Francisco Massa, et al. Pytorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems 32*, pages 8024–8035. Curran Associates, Inc., 2019.
- [21] Ning Qian. On the momentum term in gradient descent learning algorithms. *Neural Networks*, 12(1):145–151, 1999.
- [22] Shaoqing Ren, Kaiming He, Ross Girshick, and Jian Sun. Faster r-cnn: Towards real-time object detection with region proposal networks. *Advances in Neural Information Processing Systems*, 28, 2015.
- [23] Hamid Rezatofighi, Nathan Tsoi, JunYoung Gwak, Amir ... Sadeghian, and Silvio Savarese. Generalized intersection over union: A metric and a loss for bounding box regression. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 658–666, 2019.
- [24] Herbert Robbins and Sutton Monro. A stochastic approximation method. *The Annals of Mathematical Statistics*, 22(3):400–407, 1951.
- [25] Peter Rosenkranz, Pia Aumeier, and Bettina Ziegelmann. Biology and control of varroa destructor. *Journal of Invertebrate Pathology*, 103:S96–S119, 2010.
- [26] David E Rumelhart, Geoffrey E Hinton, and Ronald J Williams. Learning representations by back-propagation error. *Nature*, 323(6088):533–536, 1986.
- [27] Connor Shorten and Taghi M Khoshgoftaar. A survey on image data augmentation for deep learning. *Journal of Big Data*, 6(1):1–48, 2019.
- [28] The Thymeleaf Team. Thymeleaf: Server-side java template engine, 2024. Último acceso en mayo de 2026.
- [29] Jasper RR Uijlings, Koen EA van de Sande, Theo Gevers, and Arnold WM Smeulders. Selective search for object recognition. *International Journal of Computer Vision*, 104(2):154–171, 2013.
- [30] Guido Van Rossum and Fred L Drake. *Python 3 Reference Manual*. CreateSpace, 2009.
- [31] VMware, Inc. Spring boot official documentation, 2024. Último acceso en mayo de 2026.

- [32] W3C et al. Html5: A vocabulary and associated apis for html and xhtml, 2014. Último acceso en mayo de 2026.
- [33] Craig Walls. *Spring Boot in Action*. Manning Publications, 2016.
- [34] Rüdiger Wirth and Jochen Hipp. Crisp-dm: Towards a standard process model for data mining. In *Proceedings of the 4th International Conference on the Practical Applications of Knowledge Discovery and Data Mining*, pages 29–39. Springer-Verlag, 2000.